WordScanner-master

CSCI 340 - Data Structures and Algorithm Analysis

Data Structure: Map, String
Programming Focus: STL Map, pair, Lambda expressions, IO manipulation

Word Scanner

For this computer assignment, you are to implement an interactive C++ program to scan and process a stream of words of plain text.

Your program should start with an empty list of words. (Combinations of blanks, tabs, and newline characters to separate words.) After reading each word from the input stream, check the word for punctuation marks:

- if the first letter of the word is preceded or the last letter is followed by punctuation marks, delete punctuation marks from the word
- if a word contains punctuation marks in the middle, ignore the letters beyond these punctuation marks
- For example
  - If the word is `fish,` (fish comma), then the resulting word would be `fish`.
  - If the is `fish_net`, then the resulting word would be `fish`.
  - If the word is `"Johnson's` (the first word in a quoted string with an apostrophe), then the resulting word would be `johnson`.

Programming Notes:

1. Use a std::map to store the words, where each word is represented by a `pair<string, int>`. The first member variable of such a std::pair is named `first` and shall contain the name of the input word, and the second member variable the pair is named `second` and shall contain its frequency (number of occurrences) in the input stream.
2. In addition to the `main()` routine, implement the following subroutines in your program:
   - `void get_words(map<string, int>&)` - it gets a word from the input stream and removes its punctuation marks
   - `void print_words(const map<string, int>&)` - it prints the final list of words and their frequencies, it also prints the number of nonempty words and the number of distinct words in the input stream
   - `void clean_entry(const string&, string&)` - it cleans a word from its punctuation marks, the first argument is the original word in the input stream and the second argument contains the same word after cleaning

3. The `main()` routine calls the subroutine `get_words()` to get words from the input stream. To clean an individual word from its punctuation marks, `get_words()` calls the subroutine `clean_entry ()`.
4. You can check the size of the string returned by `clean_entry` using the member function `length()`. For string `s`, the statement `s.length()` returns the total number of characters in `s`. If the returned length is 0 (i.e., the original word contains nothing but punctuation marks), simply ignore the word and read in the next one.
5. Use a map container to store the cleaned words (but not the empty words). The statement `m[s]++` copies the string `s` into map `m` and updates its frequency. Remember a map keeps only one copy of each individual item (in sorted order) specified in its first argument key and stores its frequency in its second argument value. When key is used as an index to a map, it returns the corresponding value in the map.
6. You can implement the subroutine `clean_entry()` as follows:
    - Initialize an index variable to indicate the first character in a word, and then scan the word until an alphanumeric character is found. You can use the C library function `isalnum()` to check a character in the input string. If the character is alphanumeric, this function returns true; otherwise, it returns false. Initialize another index variable for this position and continue scanning the word until the first punctuation mark. If the word does not contain any alphanumeric characters, you must stop scanning the word when you reach the end of the word. A suitable method of implementing this approach would be to find the values of these two index variables by using the `std::find_if()` in the STL. To do this, you would invoke `std::find_if()` twice. Each with a different predicates: one of them returns `true` if a character is alphanumeric and the other one returns `true` if the character is *non-*alphanumeric. These predicates are the third argument to the two calls to `std::find_if()` and must be implemented using lamba expressions.
    - At this point you have two index variables; the first one indicates the position of the first alphanumeric character and the second one indicates the position just after the last alphanumeric character. If the word does not have any alphanumeric characters, then both index values indicate the same position.
    - Extract the cleaned version of the word from the input string by making use of the `std::string` constructor that accepts two iterators (that you will have gotten from `find_if()`) as defined in constructor (6) in std::string().
    - Finally, convert all capital letters in the cleaned word to lowercase letters. You can convert a character to lowercase by the C library function `tolower()`. Instead of using a loop for this task, use the function `std::for_each()` in the STL with a lambda expression for the unary function in the third argument.
7. Implement a subrouting called `print_words()` with the appropriate arguments to print the number of nonempty words in the input stream and the number of distinct words. You can obtain the number of elements in a map by the member function `size()`. Also print the contents of your map name of each word and its frequency (`NO_ITEMS` words per line and the words are left aligned). To allocate certain number of spaces for an output value, use the

manipulator: `setw(ITEM_WORD_WIDTH)`, and for its left alignment, use the manipulator left in your `cout` statement, separate the word by a ":" and use `setw(ITEM_COUNT_WIDTH)` to set size for count of words. In your program, use `NO_ITEMS = 4`, `ITEM_WORD_WIDTH = 14`, and `ITEM_COUNT_WIDTH = 3`.

8. Create a Makefile that builds an executable `wordscanner`, to test your program execute your program as `./wordscanner < wordscanner.refin`. This will redirect the content of `wordscanner.refin` as standard input for your code. You can compare results with standard output from assignment found in `wordscanner.refout`.

Assignment Notes:

- Include any necessary headers and add necessary global constants.
  - To use a `std::map` insert the statement: `#include <map>` in your program.
  - To use the `std::pair`, insert the spatement: `#include <utility>` in your program.
    - To use `find_if()` and `for_each()`, insert the statement: `#include <algorithm>` in your program.
- You must use a lambda expression for the predicates of `std::find_if()` and for the unary functions when using `std::for_each()`.
- You are not allowed to use any I/O functions from the C library, such as `scanf` or `printf`. Instead, use the I/O functions from the C++ library, such as `cin` or `cout`.
- Add documentation to the appropriate source files as discussed in your class.
- Prepare your Makefile (**you need to construct and add Makefile**) so that the TA only needs to invoke the command `make` to compile your source file and produce the executable file `wordscanner`. Make sure you use exactly the same file names specified here, i.e. `wordscanner.cc` and `wordscanner`, in your `Makefile`. Otherwise your submission will get 0 points.

When your program is ready for grading, ***commit*** and ***push*** your local repository to remote git classroom repository and follow the ***Assignment Submission Instructions***.